



Politechnika Warszawska

**Wydział Matematyki
i Nauk Informatycznych**

Reprezentacja Wiedzy

Procesy działań złożonych

Dokumentacja techniczna

Zespół projektowy nr 4:

Wojciech Jasiński (koordynator)

Krzysztof Gryboś

Jarosław Bieniek

Filip Filipkowski

Sebastian Rydz

Jakub Pietrzak

Sandra Adamiec

Warszawa, 11 czerwca 2026

Spis treści

1	Wprowadzenie	2
2	Architektura	2
3	Języki wejściowe — składnia plikowa	2
4	Przebieg rozwiązywania	3
5	Realizacja warunków Z1–Z7	5
6	Przykłady z obliczeniami	5
6.1	Prosty efekt — inercja i minimalizacja	5
6.2	Rosyjska ruletka — niedeterminizm przez releases	6
6.3	Konflikt w akcji złożonej — dekompozycje	6
6.4	Ramifikacja — skutek uboczny przez always	6
6.5	Niewykonalność — impossible i kwerenda executable	7
7	Obsługa błędnego wejścia	7
8	Testy	7
8.1	Testy jednostkowe	8
8.2	Testy integracyjne	8
8.3	Testy regresyjne	8
8.4	Uruchamianie i powtarzalność	9

1 Wprowadzenie

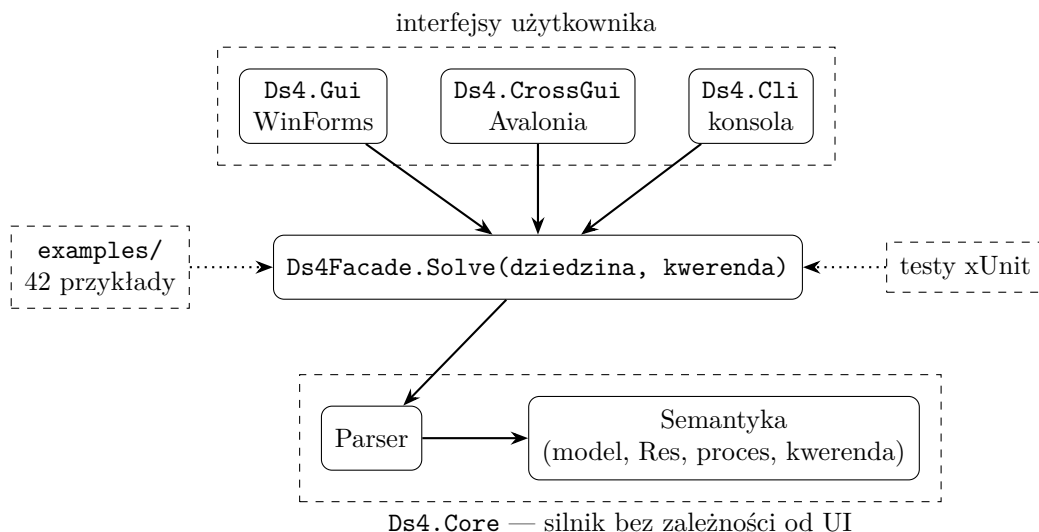
Aplikacja jest solverem dla klasy *DS4* systemów dynamicznych. Na wejściu przyjmuje tekstowy opis dziedziny (w języku akcji) oraz jedną kwerendę (w języku kwerend), buduje w pamięci model dziedziny i odpowiada **TAK** lub **NIE** na pytania Q1 (wykonywalność procesu) i Q2 (osiągnięcie celu), w wariantach *zawsze* i *kiedykolwiek*. Oprócz odpowiedzi program pokazuje krótkie uzasadnienie oraz pełne drzewo wykonań procesu (trace).

Definicje obu języków oraz ich semantyka znajdują się w *dokumentacji teoretycznej*; niniejszy dokument ich nie powtarza — opisuje wyłącznie budowę programu, przyjmowaną składnię plikową, przebieg obliczeń i testy. Tam, gdzie mowa o pojęciach formalnych (Σ , Σ_0 , Res, New, konflikty, dekompozycje), podajemy odsyłacze do odpowiednich sekcji dokumentacji teoretycznej (numeracja według jej wersji 2).

Implementacja: C# 12 / .NET 8. Silnik nie korzysta z żadnych bibliotek zewnętrznych; interfejsy graficzne używają WinForms (Windows) i Avalonii (wieloplatformowo). Testy — xUnit.

2 Architektura

System składa się z jednego silnika (*Ds4.Core*) i trzech wymiennych interfejsów. Całość spina jedna fasada.



- **Interfejsy** (WinForms / Avalonia / CLI) — tylko pola tekstowe, lista wbudowanych przykładów i przycisk „Oblicz”. Trzy frontendy korzystają z tej samej logiki.
- **Ds4Facade** — jedyne API używane przez interfejsy. Przyjmuje dwa napisy (dziedzina, kwerenda), zwraca odpowiedź TAK/NIE, uzasadnienie, rozmiary $|\Sigma|$ i $|\Sigma_0|$ oraz trace.
- **Ds4.Core** — czysty silnik: parser zamienia tekst na struktury danych, moduł semantyki buduje model i ewaluje kwerendę. Wymiana frontendu nie zmienia ani jednej linii silnika.
- **examples/** — 42 pary plików `.domain` + `.query`, ładowane do listy w GUI i CLI.
- **Testy** — xUnit; testują fasadę i moduły semantyki, w tym wszystkie przykłady względem plików `.expected`.

3 Języki wejściowe — składnia plikowa

Parser przyjmuje zapis ASCII konstrukcji zdefiniowanych w dokumentacji teoretycznej (sekcje 2.4 i 3.11). Poniższa tabela zestawia notację teoretyczną z zapisem plikowym.

Konstrukcja	Notacja teoretyczna	Zapis w pliku
efekt akcji	$A \text{ causes } \alpha \text{ if } \pi$	shoot causes !alive if loaded
zwolnienie inercji	$A \text{ releases } f \text{ if } \pi$	spin releases loaded if true
niewykonalność	impossible $A \text{ if } \pi$	impossible load if loaded
ograniczenie	always α	always p -> q
stan początkowy	initially α	initially !p and !q
fluent nieinercyjny	noninertial f	noninertial f
zdanie wartościujące	$\alpha \text{ after } \mathbb{A}_1; \dots; \mathbb{A}_n$	p after a; {b,c}
wariant obserwowalny	observable $\alpha \text{ after } \dots$	observable p after a

Formuły zapisuje się operatorami ! ($\neg$), & lub and ($\wedge$), | lub or ($\vee$), -> ($\rightarrow$), <-> ($\leftrightarrow$) oraz stałymi true/false. Komentarze zaczynają się od #. Deklaracje fluents ... i actions ... są opcjonalne — parser sam zbiera symbole występujące w zdaniach.

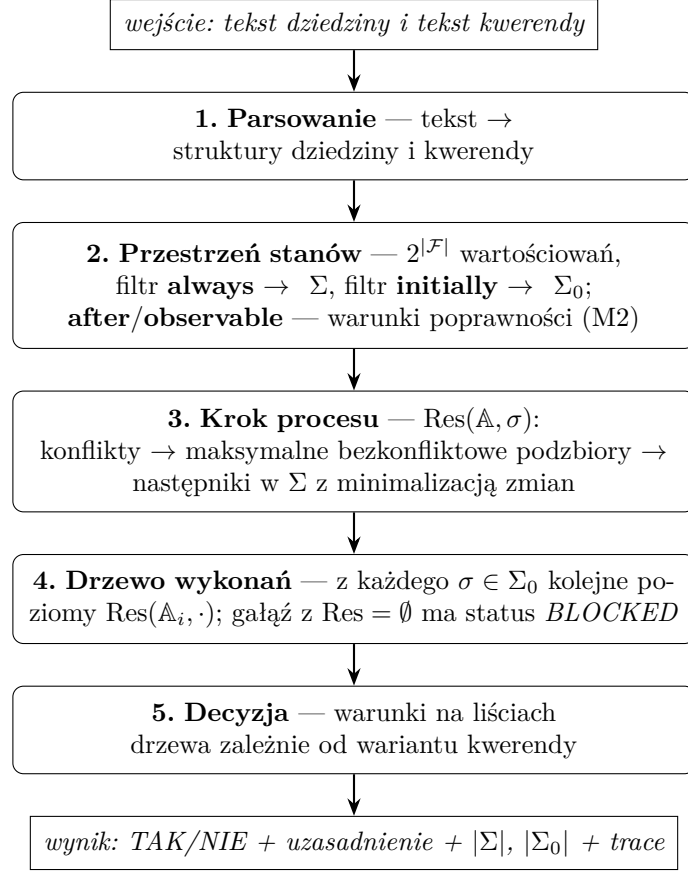
Zgodnie z dokumentacją teoretyczną **impossible** i **initially** są skrótami (odpowiednio: $A \text{ causes } \perp \text{ if } \pi$ oraz $\alpha \text{ after } \varepsilon$) — parser rozpoznaje je jako osobne słowa kluczowe, ale semantycznie traktuje tak samo jak rozwinięcia.

Kwerendy. Plik kwerendy zawiera jedno zdanie. Akcję złożoną zapisuje się w nawiasach klamrowych ($\{a, b\}$; nawiasy można pominąć dla akcji jednoelementowej), proces — jako ciąg akcji złożonych rozdzielonych średnikami. Cztery warianty pytań:

Pytanie	Wariant	Zapis w pliku
Q1: wykonywalność	zawsze	necessary executable after a; {b,c}
	kiedykolwiek	possibly executable after a; {b,c}
Q2: cel γ	zawsze	necessary !alive after spin; shoot
	kiedykolwiek	possibly !alive after spin; shoot

4 Przebieg rozwiązywania

Obliczenie przebiega w pięciu krokach. Program realizuje wprost definicje z dokumentacji teoretycznej — dla małych dziedzin (kilka–kilkanaście fluentów) pełne przeszukiwanie $2^{|\mathcal{F}|}$ stanów jest szybkie, a wierność definicjom była ważniejsza niż optymalizacje.



Krótko o każdym kroku (w nawiasach sekcje dokumentacji teoretycznej z definicjami):

1. **Parsowanie.** Z tekstu powstaje lista fluentów, akcji, reguł **causes** / **releases** / **impossible**, ograniczeń **always** / **initially** oraz proces i cel kwerendy.
2. **Przestrzeń stanów.** Generujemy wszystkie wartościowania fluentów i odsiewamy niezgodne z **always** — zostaje Σ (sekcja 2.5.1). Z Σ wybieramy stany zgodne z **initially** — zostaje Σ_0 (sekcja 2.5.2), czyli zbiór możliwych stanów początkowych przy opisie częściowym. Zbiór ten wyznaczają *wyłącznie* zdania **initially**. Zdania **after** oraz **observable after** to warunki poprawności modelu (M2): sprawdzamy je globalnie nad całym Σ_0 i *nie* usuwamy z niego pojedynczych stanów. Zdanie α **after** P wymaga, by z *każdego* stanu początkowego *każda* pełna ścieżka procesu P kończyła się stanem spełniającym α ; wariant **observable** — by tak kończyła się *przynajmniej jedna* ścieżka z *pewnego* stanu początkowego. Niespełnienie któregoś z warunków oznacza dziedzinę sprzeczną (brak modelu, $\Sigma_0 = \emptyset$).
3. **Krok procesu.** Dla akcji prostej: jeśli aktywna jest reguła **impossible**, następników brak; w przeciwnym razie następnikami są stany z Σ spełniające aktywne efekty i minimalizujące zbiór zmian New (sekcje 2.5.3–2.5.6) — tak realizujemy inercję (Z1) i skutki uboczne ograniczeń (Z6). Fluent zwolniony regułą **releases** zawsze wnosi swój literał do New, dzięki czemu oba jego wartościowania są nieporównywalne i oba przeżywają minimalizację (niedeterminizm, Z3). Dla akcji złożonej: wykrywamy konflikty par akcji w stanie σ (sekcja 3.3), wyznaczamy maksymalne bezkonfliktowe podzbiory — dekompozycje (sekcja 3.4) — i jako $\text{Res}(A, \sigma)$ bierzemy sumę wyników po dekompozycjach (sekcja 3.5).
4. **Drzewo wykonań.** Z każdego stanu początkowego rozwijamy drzewo: dzieci korzenia to $\text{Res}(A_1, \sigma)$, wnuki — $\text{Res}(A_2, \cdot)$ itd. (sekcja 3.8–3.9). Gałąź ucięta przed końcem procesu jest *zablokowana*; gałąź pełnej długości kończy się *liściem końcowym*.

5. **Decyzja** (sekcja 3.12). Kwantyfikator po stanach początkowych jest zawsze uniwersalny: przy opisie częściowym prawdziwy stan początkowy nie jest znany, więc odpowiedź musi uwzględniać każdy stan zgodny z **initially**. Warunki (we wszystkich przypadkach wymagamy $\Sigma_0 \neq \emptyset$):

- **possibly executable** — z *każdego* stanu początkowego istnieje liść końcowy;
- **possibly γ** — z *każdego* stanu początkowego istnieje liść końcowy spełniający γ ;
- **necessary executable** — brak gałęzi zablokowanych i istnieją liście końcowe;
- **necessary γ** — jak wyżej oraz *wszystkie* liście końcowe spełniają γ .

5 Realizacja warunków Z1–Z7

Treść projektu definiuje klasę *DS4* siedmioma warunkami. Poniższa tabela wskazuje, który mechanizm programu realizuje każdy z nich.

Warunek	Realizacja w programie
Z1. Prawo inercji	minimalizacja zbioru zmian New w Res: fluent nieobjęty efektem zachowuje wartość
Z2. Pełna informacja	jawna enumeracja wszystkich stanów ($2^{ \mathcal{F} }$) i wszystkich reguł dziedziny — nic nie jest domyślne
Z3. Niedeterminizm działań	releases (oba wartościowania zwolnionego fluentu przeżywają minimalizację) oraz suma wyników po dekompozycjach akcji złożonej
Z4. Warunek wstępny i wynikowy	reguły A causes α if π ; gdy π nie zachodzi, reguła jest nieaktywna (efekt pusty)
Z5. Niewykonalność	impossible A if π — Res = $\emptyset$, gałąź drzewa dostaje status BLOCKED
Z6. Warunki integralności	filtr always na Σ ; następniki wybierane wyłącznie z Σ , więc ograniczenia wymuszają skutki uboczne (ramifikacja)
Z7. Opis częściowy	Σ_0 jako zbiór <i>wszystkich</i> stanów zgodnych z initially ; after/observable to warunki poprawności (M2), nie filtry na Σ_0 ; uniwersalny kwantyfikator po Σ_0 w obu rodzajach kwerend

6 Przykłady z obliczeniami

Pięć przykładów z wbudowanego zestawu; każdy pokazuje inny mechanizm. Zapisy dziedziny i kwerend są dosłownie tym, co przyjmuje program.

6.1 Prosty efekt — inercja i minimalizacja

```
initially !p
make causes p if true
```

Kwerenda: **necessary p after make**

- $\mathcal{F} = \{p\}$, brak **always**, więc $\Sigma = \{\{p\}, \{\neg p\}\}$; **initially !p** daje $\Sigma_0 = \{\{\neg p\}\}$.
- Res(*make*, $\neg p$): aktywny efekt p ; kandydaci spełniający p : tylko $\{p\}$, New = $\{p\}$ — wynik $\{\{p\}\}$.
- Jedyne liść końcowy spełnia p , gałęzi zablokowanych brak. **Odpowiedź: TAK.**

6.2 Rosyjska ruletka — niedeterminizm przez releases

```

fluents loaded, alive
actions spin, shoot, load
initially alive
spin releases loaded if true
shoot causes !alive if loaded
impossible load if loaded
load causes loaded if !loaded

```

Kwerendy: possibly !alive after spin; shoot oraz necessary !alive after spin; shoot

- $|\Sigma| = 4$; **initially alive** ogranicza tylko *alive*, więc Σ_0 ma dwa stany: $\{l, a\}$ i $\{\neg l, a\}$ (opis częściowy).
- $\text{Res}(\text{spin}, \sigma)$: brak efektów **causes**, zwolniony *loaded* — oba wartościowania *loaded* przeżywają minimalizację, więc z każdego stanu początkowego dostajemy $\{l, a\}$ i $\{\neg l, a\}$.
- $\text{Res}(\text{shoot}, \cdot)$: z $\{l, a\}$ aktywny efekt $\neg \text{alive}$ daje $\{l, \neg a\}$; z $\{\neg l, a\}$ żadna reguła nie jest aktywna — inercja zostawia $\{\neg l, a\}$.
- Z każdego z dwóch stanów początkowych istnieje więc ścieżka kończąca się $\{l, \neg a\} \models \neg \text{alive}$ — ale wśród liści jest też $\{\neg l, a\}$, więc nie wszystkie ścieżki kończą się celem. **Odpowiedzi: possibly — TAK, necessary — NIE.**

6.3 Konflikt w akcji złożonej — dekompozycje

```

initially !p
makeP causes p if true
clearP causes !p if true

```

Kwerenda: necessary p after {makeP, clearP}

- $\Sigma = \{\{p\}, \{\neg p\}\}$, $\Sigma_0 = \{\{\neg p\}\}$.
- W stanie $\neg p$ akcje *makeP* i *clearP* wymuszają literały przeciwne (p i $\neg p$) — konflikt. Maksymalne bezkonfliktowe podzbiory: $\{makeP\}$ i $\{clearP\}$.
- $\text{Res}(\{makeP, clearP\}, \neg p) = \text{Res}(makeP, \neg p) \cup \text{Res}(clearP, \neg p) = \{\{p\}, \{\neg p\}\}$.
- Liść $\{\neg p\}$ nie spełnia p . **Odpowiedź: NIE.**

6.4 Ramifikacja — skutek uboczny przez always

```

always p -> q
initially !p and !q
makeP causes p if true

```

Kwerenda: necessary q after makeP

- Z czterech wartościowań ograniczenie **always p -> q** odrzuca $\{p, \neg q\}$, więc $|\Sigma| = 3$; $\Sigma_0 = \{\{\neg p, \neg q\}\}$.
- $\text{Res}(makeP, \{\neg p, \neg q\})$: aktywny efekt p ; jedynym stanem w Σ spełniającym p jest $\{p, q\}$ — fluent q zmienia się jako skutek uboczny, bo stan $\{p, \neg q\}$ nie należy do Σ .
- Jedyny liść spełnia q . **Odpowiedź: TAK.**

6.5 Niewykonalność — impossible i kwerenda executable

```
fluents loaded
actions load
initially loaded
impossible load if loaded
load causes loaded if !loaded
```

Kwerenda: possibly executable after load

- $\Sigma = \{\{l\}, \{\neg l\}\}$, $\Sigma_0 = \{\{l\}\}$.
- $\text{Res}(\text{load}, \{l\})$: aktywna jest reguła **impossible**, więc następników brak — jedyna gałąź drzewa jest zablokowana już na pierwszym kroku.
- Ze stanu początkowego nie istnieje żadna pełna ścieżka. **Odpowiedź: NIE.**

7 Obsługa błędnego wejścia

Fasada nie rzuca wyjątków do interfejsu — każdy problem z wejściem kończy się czytelnym komunikatem zamiast odpowiedzi:

- **błąd składni** w dziedzinie lub kwerendzie — zwracany jest komunikat parsera wskazujący niepoprawne zdanie;
- **sprzeczne ograniczenia always** — żaden stan ich nie spełnia ($\Sigma = \emptyset$); program zgłasza: „Model jest pusty: ograniczenia always są sprzeczne”;
- **sprzeczny opis początkowy** — $\Sigma_0 = \emptyset$; wszystkie kwerendy mają wtedy odpowiedź NIE, a w wyniku widać $|\Sigma_0| = 0$, co wskazuje przyczynę;
- **bardzo duże drzewo wykonań** — trace jest ucinany po około 60 tys. znaków z jawną adnotacją, żeby nie blokować GUI; odpowiedź i uzasadnienie są liczone zawsze na pełnym drzewie.

8 Testy

Poprawność silnika weryfikujemy zautomatyzowanym zestawem testów (framework xUnit, projekt `Ds4.Tests`). Zestaw liczy **82 metody testowe**, które po rozwinięciu testów parametryzowanych (`[Theory]`) dają **137 przypadków testowych** — wszystkie zielone, czas wykonania poniżej 0,1 s. Testy działają w trybie wsadowym (bez interfejsu graficznego) wprost na czystym silniku `Ds4.Core`, w większości przez tę samą fasadę `Ds4Facade.Solve`, której używają wszystkie frontendy. Silnik nie ma zależności zewnętrznych ani żadnego źródła losowości, więc każdy przebieg jest w pełni *powtarzalny*.

Przyjęliśmy trójwarstwową strategię testowania, od najdrobniejszych jednostek do całych scenariuszy:

- **testy jednostkowe** — sprawdzają pojedyncze moduły semantyki w izolacji (parser, formuły, model, Res, dekompozycje, drzewo wykonań);
- **testy integracyjne** — sprawdzają cały tor obliczeń „od tekstu do TAK/NIE” na zestawie wbudowanych przykładów;
- **testy regresyjne** — utrwalają poprawione interpretacje semantyczne, aby raz naprawiony błąd nie wrócił.

8.1 Testy jednostkowe

Każdy moduł silnika ma własny plik testowy. Testy budują minimalną dziedzinę z napisu (pomocnik `TestData.BuildModel/Solve`) i sprawdzają jeden aspekt zachowania, dzięki czemu ewentualna regresja od razu wskazuje winny moduł.

Plik testowy	Co sprawdza	Testów
<code>ParserTests</code>	składnia dziedziny, kwerendy, procesu i formuł; odrzucanie błędnych zdań	12
<code>FormulaTests</code>	wartościowanie formuł, priorytety operatorów, postać DNF, prawa de Morgana	7
<code>ModelTests</code>	równość stanów (niezależna od kolejności i wielkości liter), wnioskowanie fluentów i akcji	4
<code>StateAndModelBuilderTests</code>	generacja Σ ($2^{ F }$ wartościowań, filtr always) i Σ_0 (initially); wykrywanie sprzecznych ograniczeń	6
<code>SimpleActionEngineTests</code>	Res akcji prostej: causes , releases (niedeterminizm), impossible , inercja, ramifikacje, akcja pusta	8
<code>CompositeActionTests</code>	wykrywanie konfliktów, generacja dekompozycji, wykonanie akcji złożonej	9
<code>ProcessAndQueryTests</code>	ewaluacja procesu, drzewo wykonań, cztery warianty kwerend (<i>possibly/necessary</i> $\times$ wykonalność/cel)	9

8.2 Testy integracyjne

Najszerszą warstwę stanowi **42 wbudowanych przykładów** w katalogu `app/examples/`. Każdy przykład to trójka plików o wspólnej nazwie: *id.domain* (dziedzina), *id.query* (kwerenda) oraz *id.expected* (oczekiwana odpowiedź: pojedyncze słowo TAK albo NIE). Test parametryzowany ([Theory] + [MemberData], źródło `TestData.ExampleFiles`) ładuje każdą trójkę, rozwiązuje ją przez `Ds4Facade.Solve` i porównuje wynik z plikiem `.expected`. Te same przykłady zasilają listę w GUI i CLI, więc test pełni podwójną rolę: weryfikuje silnik oraz gwarantuje, że każdy przykład pokazywany użytkownikowi daje deklarowaną odpowiedź.

Zestaw przykładów dzieli się na **24 z odpowiedzią TAK** i **18 z NIE**, a nazwa każdego koduje jego pochodzenie w schemacie `{tak|nie}_{tex|extra|bugfix}_nn_opis`:

Rodzina	Pochodzenie	TAK	NIE
<code>tex</code>	scenariusze z wykładu i dokumentacji teoretycznej (rosyjska ruletka, producent i konsumenci, dwa przełączniki, dwóch robotników)	13	7
<code>extra</code>	przykłady uzupełniające pokrycie konstrukcji języka	10	10
<code>bugfix</code>	utrwalenie błędów wykrytych w rozwoju (np. warunkowe causes)	1	1
Razem		24	18

Spójności samego zestawu pilnują dodatkowe testy: każda dziedzina musi mieć parę `.query` + `.expected`, a pliki `.expected` mogą zawierać wyłącznie TAK lub NIE.

8.3 Testy regresyjne

Część testów powstała w odpowiedzi na konkretne, naprawione błędy interpretacji semantyki — utrwalają one decyzje opisane w *dokumentacji teoretycznej* (wersja 2) i chronią przed ich cofnięciem:

- Σ_0 **nie jest zawężane** przez zdania **after** ani **observable**. Zdanie **after** jest warunkiem poprawności modelu (M2): jego naruszenie unieważnia całą dziedzinę ($\Sigma_0 = \emptyset$), a nie usuwa pojedynczego stanu

(`After_Assertion_Is_Validity_Condition_Not_A_Filter...`). Zdanie **observable** jest egzystencjalne i również nie usuwa stanów początkowych, nawet jeśli z niektórych celu nie da się osiągnąć (`Observable_After_Does_Not_Trim_Initial_States...`).

- **possibly** kwantyfikuje uniwersalnie po Σ_0 (dla *każdego* stanu początkowego istnieje ścieżka do celu), a nie egzystencjalnie. Plik `PossiblySemanticsTests` (7 testów) sprawdza m.in., że brak ścieżki z jednego stanu daje odpowiedź NIE, oraz że dla jednoelementowego Σ_0 wynik pokrywa się ze starą, egzystencjalną semantyką.
- **warunkowe causes** (A causes q if p) działają tylko na gałęziach, gdzie warunek p zachodzi (`ConditionalCauseRegressionTests`, 6 testów).

8.4 Uruchamianie i powtarzalność

Cały zestaw uruchamia polecenie `dotnet test` w katalogu `app/` (na Windows także skrypt `RUN_TESTS.bat`). Wymagane jest jedynie .NET 8; brak zależności zewnętrznych sprawia, że wynik jest identyczny na każdej maszynie. Przypisanie testów do członków zespołu znajduje się w osobnym dokumencie „Wkład osobowy”.